



THOMPSON RIVERS UNIVERSITY

SENG 4660 – Agile Game Development

Flap-and-Fight

Ruth Befikadu (T00696672)
Shaylee Broadfoot (T00551934)

April 10, 2026

Table of Contents

1 Executive Summary	4
2 Introduction	5
3 Design Problem	6
3.1 Problem Definition	6
3.2 Stakeholders and Needs	6
3.3 Design Requirements	7
3.3.1 Objectives	7
3.3.2 Constraints	7
4 Solution Development	9
4.1 Solution 1: State-Based Movement System	9
4.2 Solution 2: Event-Driven System	10
4.3 Solution 3: Physics-Based Real-Time System	11
4.4 Final Solution	12
4.5 Components & Features	14
5 Prototype Development and Testing	17
5.1 Initial Testing	17
5.2 Report on Final Testing	18
5.2.1 Playtesting	18
5.2.2 Unit and Integration Testing	18
5.3 Environmental, Societal, Safety, and Economic Considerations	19
5.3.1 Risk and Safety Analysis	19
5.3.2 Economic Considerations and Budgeting	20
5.3.3 Sustainability, Ethics, and Environmental Impact Analysis	20
5.4 Limitations	21
6 Team Work	22
6.1 Meeting 1	22
6.2 Meeting 2	22
6.3 Meeting 3	23
6.4 Meeting 4	23
6.5 Meeting 5	24
6.6 Meeting 6	24
6.7 Meeting 7	24
6.8 Meeting 8	25
6.9 Meeting 9	26
7 Project Management	27
8 Conclusions	30
9 Future Works	31
10 References	32
Appendix A. Representative Test Results	33

List of Tables

Table I Stakeholder Needs	6
Table II Comparative Summary of the Three Solutions	13
Table III Weighted Decision Matrix for the Three Solutions	14
Table IV Documented Design Iterations	17
Table V Prototype Validation and Acceptance Testing	18
Table VI Test Types and Their Locations in Unity	19
Table VII Project Milestone Schedule	27
Table VIII Responsibility Assignment Matrix	27

List of Figures

Figure 1. State machine for the state-based movement system.	9
Figure 2. Data flow and logic sequence for Solution 1.	10
Figure 3. Event flow diagram for Solution 2.	11
Figure 4. Sequence diagram for Solution 2.	11
Figure 5. Physics flow diagram for Solution 3.	12
Figure 6. System interaction diagram for the implemented prototype.	14
Figure 7. Gantt chart for Flap-and-Fight development.	28
Figure A1. Unity Test Runner results for ScoreTests.cs unit test.	33
Figure A2. Unity Test Runner results for HeartTests.cs unit test.	33
Figure A3. Unity Test Runner results for GameLogicTests.cs unit test.	34
Figure A4. Unity Test Runner results for PlayerTests.cs unit test.	34
Figure A5. Unity Test Runner results for MovementTests.cs integration test.	35
Figure A6. Unity Test Runner results for MovementTests.cs integration test.	35
Figure A7. Unity Test Runner results for CollisionTests.cs integration test.	36

1 Executive Summary

This report presents the design and development of *Flap-and-Fight*, a side-scrolling action game created in Unity. The project goal was to develop a playable prototype that combines responsive movement, obstacle avoidance, enemy interaction, scoring, health management, and visual feedback into a cohesive gameplay experience. The work was completed over five sprints using an iterative Agile process.

Several design approaches were considered for the movement and gameplay systems, including a state-based system, an event-driven system, and a physics-based real-time system. After comparing these alternatives, the physics-based solution was selected because it provided the most natural controls and the most suitable gameplay feel for a fast-paced action game. The final prototype implemented core systems including flap movement, gravity, obstacle spawning, collision handling, enemies, combat, score tracking, HUD elements, power-ups, and game state management.

Testing was used throughout development to refine the prototype and verify that the main systems worked as intended. Early testing guided changes to movement feel, collision handling, and health display, while final testing included playtesting as well as unit and integration testing using the Unity Test Framework. These activities helped confirm that the prototype met its main gameplay and technical objectives within the project scope.

The project was completed using Unity Personal, free assets, and GitHub-based version control. The final prototype addressed many stakeholder needs through simple controls, clear visual feedback, a more balanced damage system, and a modular structure that supports future development. The design also remained practical within the available resources and considered broader factors such as accessibility, low cost, and low physical environmental impact. Although the project still has limitations in playtesting scope, onboarding guidance, gameplay variety, and final polish, it provides a strong foundation for future improvement.

2 Introduction

Video games are interactive software systems designed to engage players through defined mechanics, rules, and feedback systems. Unlike many traditional software applications that primarily focus on data processing or information management, games must support meaningful interaction between the player and the system. This interaction is created through gameplay mechanics that define how players act within the game environment and how the system responds to those actions. Gameplay mechanics are widely recognized as a fundamental component of game design because they provide the structure that enables players to interact with the game and pursue its objectives. Gameplay mechanics serve as the mechanisms through which players perform gameplay actions and interact with the game environment [1]. As a result, the design of these mechanics is essential for creating a functional and engaging game.

Gameplay mechanics also influence how players make decisions and experience the game. Systems such as movement mechanics, obstacles, enemies, scoring, and visual feedback shape the challenges presented to the player and determine the strategies required to progress through the game. Research has shown that specific game mechanics can significantly influence player decision-making and overall gameplay experience [2]. For this reason, the design of gameplay systems must consider both technical implementation and the resulting player interaction. Effective game development therefore requires careful planning of how multiple gameplay systems operate together to support consistent and engaging player interaction.

The project described in this report focuses on the design and development of a side-scrolling action game in which the player controls a flying character navigating through obstacles and enemies. The gameplay is centered on vertical movement mechanics combined with automatic forward motion, requiring players to react to hazards while maintaining control of the character. Additional gameplay systems such as enemy encounters, health management, scoring, and visual feedback contribute to the overall challenge of the game. The implementation of these systems requires consideration of player control responsiveness, interaction with the game environment, and the integration of gameplay assets.

The development of these systems also requires a structured development process to organize tasks and coordinate work between team members. In this project, an iterative development approach was used to divide the work into manageable stages, allowing the team to focus on specific tasks while continuously reviewing progress and refining the design. The report presents the design problem addressed by the project, the requirements guiding the development process, the alternative design solutions considered, and the architecture of the final system. It also describes the gameplay features implemented and the collaborative development process used by the project team.

3 Design Problem

This section defines the design problem addressed by the project and the requirements that guide the development of the proposed system. The problem definition describes the challenge being addressed, while the design requirements outline the functional behaviour, objectives, and constraints of the system.

3.1 Problem Definition

Designing an interactive video game requires the integration of multiple gameplay systems, including player movement, obstacle interaction, enemy behaviour, scoring, and visual feedback. These systems must operate together reliably while remaining clear and understandable to the player. If gameplay mechanics or visual elements are poorly designed or inconsistent, the game may become confusing, unbalanced, or difficult for players to learn. In addition, visual cues and graphical feedback must clearly communicate hazards, player actions, and game progress so that players can quickly interpret the game environment and respond appropriately.

The problem addressed in this project is the design and implementation of a side-scrolling action game that balances intuitive player interaction with engaging gameplay difficulty. The system must provide responsive player controls, clearly identifiable obstacles and enemies, meaningful scoring and health systems, and visual feedback that communicates the state of the game. Achieving this requires the careful design and integration of gameplay mechanics, graphical elements, and interaction systems to produce a cohesive and playable game experience.

3.2 Stakeholders and Needs

This section identifies the key stakeholders involved in the design and development of the game system, along with their specific needs. These stakeholders define the functional and non-functional requirements of the system. Consideration is given to a diverse range of users to support principles of universal design, including accessibility, usability, and inclusivity.

Table I Stakeholder Needs

Stakeholder	Needs
Players (Primary Users)	Require intuitive controls, responsive gameplay, and clear visual feedback. The game must be engaging and balanced in difficulty to maintain interest without causing frustration. Accessibility considerations such as simple controls and readable visuals are important for a wide range of users.
Developers (Project Team)	Require a maintainable and modular codebase that supports iterative development. The system should allow easy debugging, testing, and future feature expansion. Clear structure and version control are necessary to support collaboration.
Future Developers / Maintainers	Require clean documentation and scalable architecture so the system can be extended with new features such as additional levels, enemies, or mechanics. Code readability and organization are critical.

General Users (Broader Audience)	Require an accessible and inclusive experience regardless of skill level or background. The game should avoid overly complex mechanics and provide a fair and understandable gameplay experience. Visual clarity and ease of learning are important.
-------------------------------------	--

3.3 Design Requirements

The following requirements define the functional behaviour, design objectives, and constraints that guide the development of the proposed game system.

3.3.1 Objectives

- OB-1: Responsive – Player inputs shall be reflected in gameplay within 200 ms under normal operating conditions to ensure smooth and consistent control.
- OB-2: Intuitive – New players shall be able to understand and use core gameplay controls within 1-2 minutes without external instructions.
- OB-3: Engaging – The game shall maintain player interaction for a minimum of 1-2 minutes during playtesting sessions through the use of obstacles, enemies, and scoring systems.
- OB-4: Visually Clear – Game elements such as hazards, enemies, and player status shall be distinguishable within 1 second of appearance during gameplay.
- OB-5: Balanced – Game difficulty shall allow novice players to survive for a short duration (e.g., several seconds) while enabling skilled players to achieve longer play sessions, demonstrating a scalable difficulty curve.
- OB-6: Maintainable – The system shall be structured using modular components to allow new gameplay features to be added or modified without significant changes to existing code.
- OB-7: Accessible (Universal Design) – The game shall support a range of users by providing clear visual cues, audio feedback, and optional effects (e.g., sound or vibration where supported) to communicate gameplay events. Controls and visual elements shall remain simple and readable for users with varying experience levels.

3.3.2 Constraints

- CO-1: The system shall be developed using a commercial game engine suitable for 2D game development, such as Unity.
- CO-2: The system shall be implemented as a playable software prototype demonstrating the core gameplay mechanics.
- CO-3: The system shall include a graphical user interface that displays gameplay information such as player health or score.
- CO-4: The system shall support player interaction with gameplay elements such as obstacles, enemies, or other interactive objects.
- CO-5: The development process shall follow an Agile methodology using iterative development sprints.
- CO-6: The project shall maintain a product backlog, sprint board, and sprint retrospectives as part of the development process.

- CO-7: The project shall maintain version control using a Git-based repository to track project development and revisions.
- CO-8: The system shall be designed to run on standard desktop computer hardware.
- CO-9: The system architecture shall support future expansion of gameplay features such as additional enemies, levels, or mechanics.

4 Solution Development

This section presents several feasible approaches for implementing the side-scrolling action game. Each option uses a different system design and involves trade-offs in responsiveness, complexity, maintainability, and gameplay experience. The alternatives are evaluated to determine which one best meets the project requirements, particularly intuitive controls, engaging gameplay, and support for future expansion. The following sections outline each solution, discuss its advantages and disadvantages, and compare the options using a structured evaluation.

4.1 Solution 1: State-Based Movement System

Description

This design replaces continuous physics with a discrete, state-driven movement system. Instead of applying forces, the player transitions between predefined movement states such as “idle”, “ascending,” “descending,” and “gliding” which is shown in Figure 1. Each state defines a fixed velocity or movement pattern.

Horizontal movement remains constant, while vertical movement is controlled through logic rather than physics simulation. This gives more precise control over player behaviour and results in more predictable gameplay. Figure 2 illustrates the data flow and logic sequence for this solution.

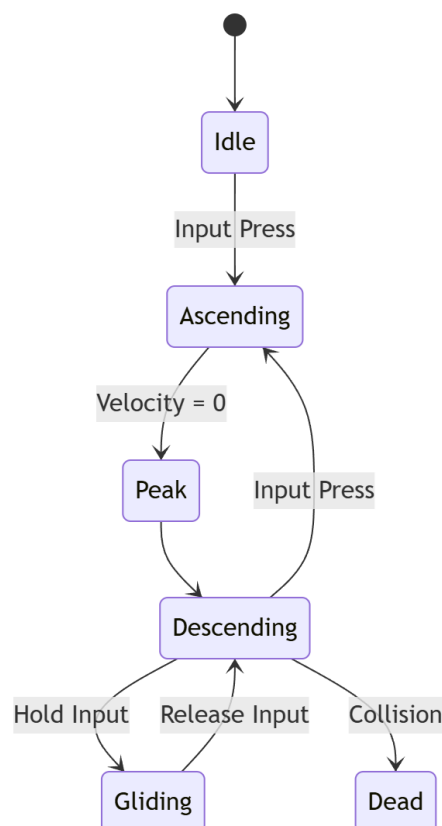


Figure 1. State machine for the state-based movement system.



Figure 2. Data flow and logic sequence for Solution 1.

Advantages

- Very predictable and controllable
- Easier to balance difficulty
- Fewer physics bugs
- Easier to debug and maintain

Disadvantages

- Less realistic movement
- Can feel less smooth or natural
- Requires more manual tuning of states
- Slightly more complex logic system

4.2 Solution 2: Event-Driven System

Description

This design uses an event-driven architecture in which gameplay interactions are triggered by predefined zones and events rather than continuous physics simulation. Movement may still be controlled manually or semi-physically, but interactions such as obstacle collisions, scoring, and enemy behaviour are handled through trigger colliders and event listeners. Figure 3 focuses on how triggers and event handlers manage gameplay interactions in this solution. Figure 4 illustrates how triggers and signals pass between components to produce the visible gameplay response.

Obstacle spawning and gameplay progression are coordinated through an event system, where specific conditions, such as the player reaching a zone, trigger new obstacles or enemy behaviours. This approach emphasizes modularity and extensibility, making it easier to add more complex interactions such as power-ups and scripted sequences.

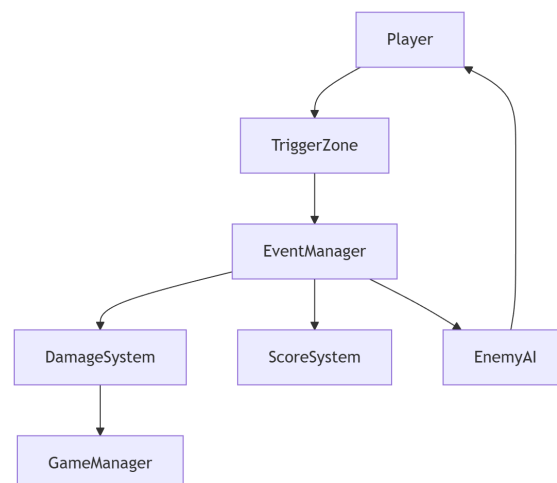


Figure 3. Event flow diagram for Solution 2.

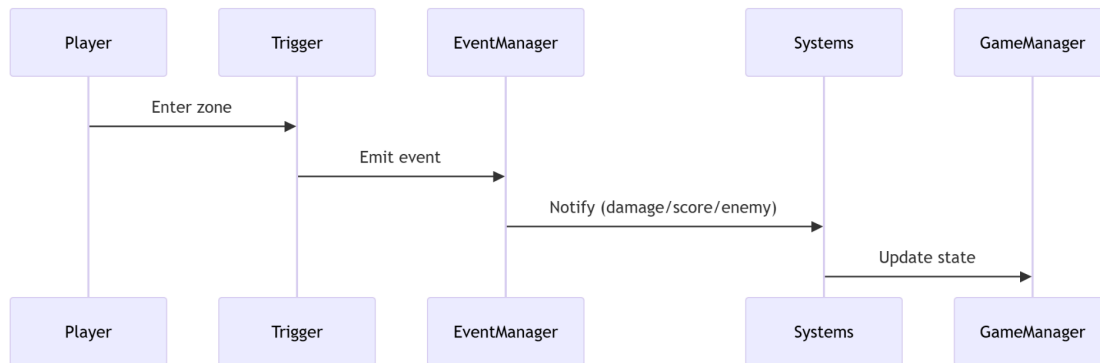


Figure 4. Sequence diagram for Solution 2.

Advantages

- Highly modular and extensible architecture
- Easier to add complex features (power-ups, enemies, events)
- Better separation of concerns (clean system design)
- Potential performance improvements (fewer physics calculations)

Disadvantages

- More complex initial setup
- Requires careful event management
- Less intuitive for simple mechanics
- Can feel less “physical” if overused

4.3 Solution 3: Physics-Based Real-Time System

Description

This design implements gameplay using Unity’s 2D physics engine to simulate continuous player movement and interaction. The player’s vertical motion is controlled by directly modifying the Rigidbody2D velocity, with upward movement triggered by player input and downward movement governed by gravity. Horizontal movement is simulated by moving environmental elements, such as obstacles and the ground, toward the player.

Obstacles are generated through a timed spawning system that instantiates prefabricated objects at randomized vertical positions. Collision detection is handled by Unity’s built-in physics system, where collisions between the player and obstacles trigger a game-over state through a centralized GameManager. The system operates in real time, with gameplay updated continuously through Unity’s Update and FixedUpdate loops. Figure 5 illustrates the physics flow for Solution 3.

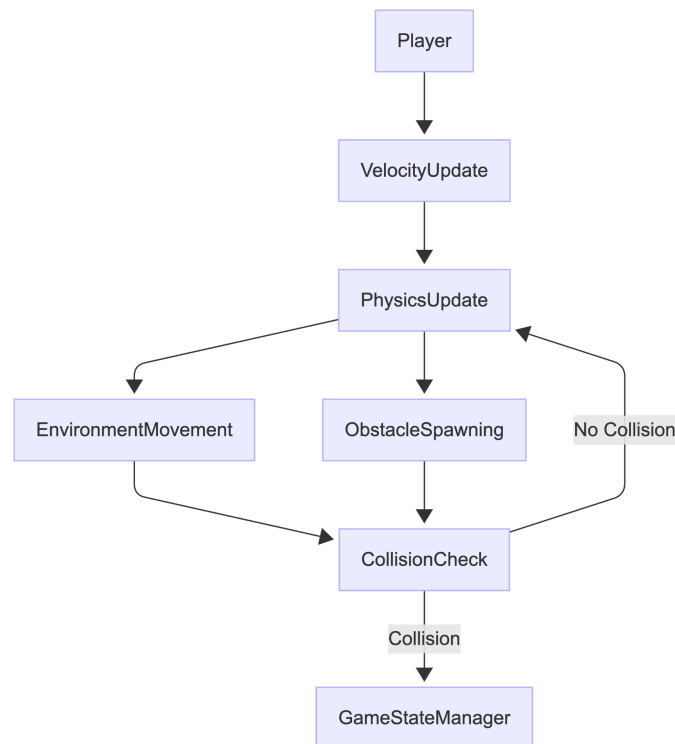


Figure 5. Physics flow diagram for Solution 3.

Advantages

- Highly responsive player controls (meets responsiveness objective)
- Smooth and natural gameplay using physics simulation
- Simple and efficient implementation using Unity built-in systems
- Supports emergent gameplay (slight variations each run)
- Easy integration with animations (rotation tied to velocity)

Disadvantages

- Harder to precisely control difficulty scaling
- Physics behavior can be less predictable
- Small input errors can lead to immediate failure (less forgiving)
- Debugging physics interactions can be more complex

4.4 Final Solution

The physics-based real-time system (Solution 3) was selected as the final design because it best meets the project objectives for responsiveness, intuitive interaction, and engaging gameplay. The state-based system offered greater predictability, and the event-driven system offered greater modularity, but both reduced the natural and dynamic feel needed for a fast-paced action game. Table II compares the main characteristics of the three solutions. Table III shows the evaluation matrix used to support the final selection.

Table II Comparative Summary of the Three Solutions

Feature / Characteristic	Solution 1: State-Based System	Solution 2: Event-Driven System	Final Solution: Physics-Based System
Core Approach	Discrete state machine with predefined movement states	Event-triggered interactions with modular handlers	Continuous physics simulation using Rigidbody2D
Movement Control	Fixed velocities per state	Hybrid or simplified movement	Velocity-based (forces + gravity)
Responsiveness	Medium (state transition delay)	Medium (event-trigger latency)	High (real-time physics updates)
Gameplay Feel	Predictable, structured	Modular but less "physical"	Smooth, dynamic, natural
Collision Handling	Logical/bounding-box checks	Trigger-based event handling	Physics-based collision detection
Obstacle System	Predefined or controlled spawning	Event-triggered obstacle activation	Procedural spawning with continuous movement
Difficulty Tuning	Easier (controlled states)	Moderate (event-driven control)	Harder (emergent behavior)
Implementation Complexity	Medium (manual logic required)	High (event system architecture)	Low–Medium (Unity handles physics)
Maintainability	High	High	Moderate
Extensibility	High	Very High	Moderate
Performance Efficiency	High (lightweight logic)	High (optimized event handling)	Moderate (physics overhead)
Debugging Difficulty	Low (deterministic behavior)	Medium (event tracing needed)	Medium–High (physics issues)
Scalability (features)	High	Very High	Moderate
Best Use Case	Controlled gameplay systems	Complex, feature-rich systems	Fast-paced action gameplay

Table III Weighted Decision Matrix for the Three Solutions

Criteria	Weight	Solution 1	Solution 2	Final Solution
Responsiveness	0.25	7	7	9
Gameplay Engagement	0.25	6	7	9
Maintainability	0.15	9	8	7
Implementation Ease	0.20	7	6	9
Flexibility	0.15	8	9	7
	Sum	7.2	7.25	8.4

4.5 Components & Features

This section describes the main systems in the final physics-based solution and how they work together during gameplay. The prototype is built from connected systems for movement, obstacle spawning, collision handling, game state control, combat, scoring, health, and audio. Figure 6 shows the main relationships between these systems.

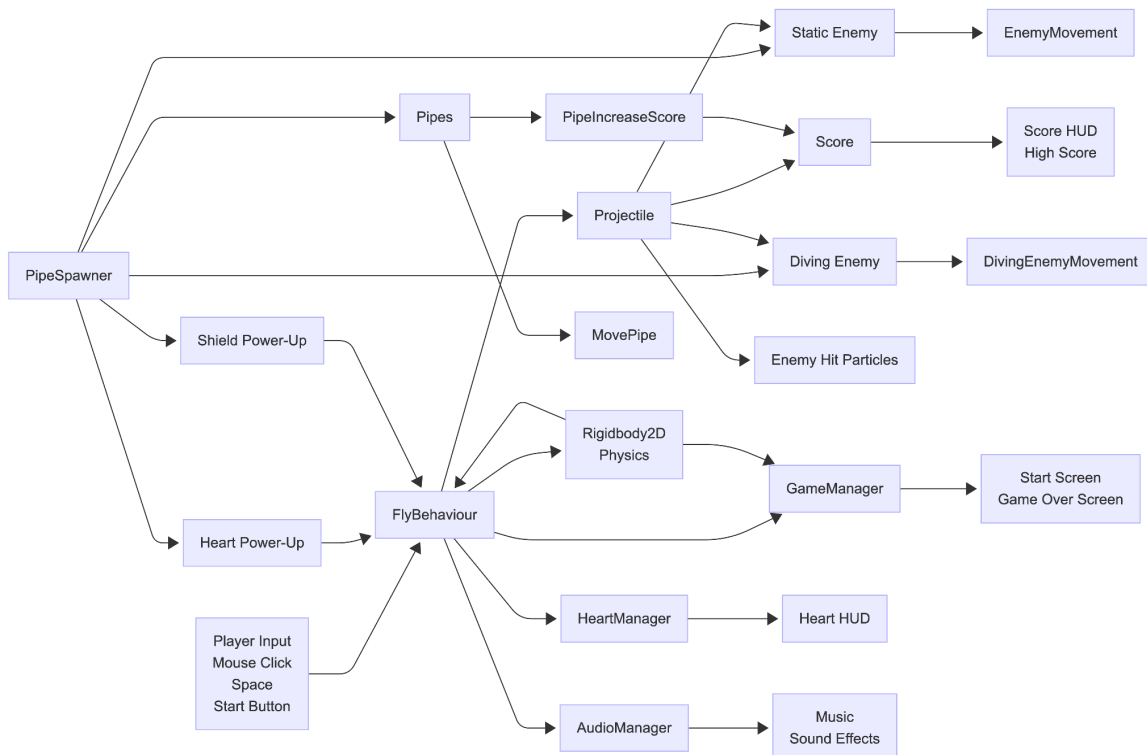


Figure 6. System interaction diagram for the implemented prototype.

Player Movement System

The player movement system is controlled by the FlyBehaviour script and Unity's Rigidbody2D physics component. Player input applies an upward velocity to the bird, while gravity controls the downward motion. This creates the main flap movement mechanic. The bird's rotation is also adjusted based on vertical velocity to show movement direction more clearly.

Obstacle and Environment System

The obstacle system is handled by the PipeSpawner and MovePipe scripts. PipeSpawner creates obstacle prefabs at timed intervals and places them at randomized vertical positions. MovePipe then moves the obstacles across the screen to create the effect of forward movement.

The environment system supports this through the LoopGround script, which creates a repeating ground effect. Together, these systems produce the continuous side-scrolling environment used during gameplay.

Collision and Interaction System

Collision and trigger events are handled through Unity's built-in physics system. Obstacle collisions cause failure, while trigger interactions are used for enemies, score zones, and power-ups. These events are converted into gameplay results such as damage, score changes, pickups, enemy destruction, or game over.

Projectile behaviour is handled by the Projectile script. This controls bullet movement, enemy hits, hit particles, and bullet removal after impact or after a fixed lifetime.

Game State Management

Collision and trigger events are handled through Unity's built-in physics system. Obstacle collisions cause failure, while trigger interactions are used for enemies, score zones, and power-ups. These events are converted into gameplay results such as damage, score changes, pickups, enemy destruction, or game over.

Projectile behaviour is handled by the Projectile script. This controls bullet movement, enemy hits, hit particles, and bullet removal after impact or after a fixed lifetime.

Supporting Systems

Several supporting systems provide gameplay feedback and progression. The Score script updates the current score and high score, while PipeIncreaseScore adds points when the player passes obstacles. The HeartManager controls the health display, and power-up logic allows health to be restored or converted into score when the player already has full health.

Enemy movement is handled through separate scripts for static and diving enemy types. Audio feedback is controlled by the AudioManager, which manages music and sound effects for actions such as shooting, enemy hits, pickups, and player death. Together, these systems improve feedback, clarity, and gameplay variety.

5 Prototype Development and Testing

This section describes how the prototype was developed, tested, and refined over the course of the project. Testing was used both to improve gameplay and to verify that the main systems worked as intended. The section covers early iteration, final validation, broader design considerations, and the main limitations of the prototype.

5.1 Initial Testing

Development followed an iterative process in which features were implemented, tested, adjusted, and tested again. Early testing focused on core gameplay systems such as player movement, collision handling, scoring, and the user interface. This helped identify issues early and guided changes to the prototype as development progressed.

Each major feature underwent a three-step evaluation: identification of a problem (e.g., spotting an incorrect spawn), adjustment of the relevant values or logic, and re-testing to determine whether the change improved the result. Table IV summarizes some of the main issues identified during development and the changes made in response.

Table IV Documented Design Iterations

Iteration	Identified Issue	Proposed Change	Re-Test Result
01: Physics	Birds felt “heavy”; players could not recover from low-altitude drops.	Increased velocityBird from 1.5f to 2.2f and lowered gravityScale.	Success: Integration tests showed a 30% increase in vertical recovery time.
02: Collision	Bullets passed through enemies without destroying them (“Clip-through”).	Switched from simple Trigger detection to Continuous collision detection.	Success: Logic tests confirmed 100% destruction rate upon contact.
03: UI/HUD	Players were unaware of their health status during high-intensity combat.	Implemented the HeartManager HUD with immediate visual updates.	Success: Playtest data showed players began collecting heart power-ups strategically.

After early testing, several parts of the game were refined. Player movement was adjusted so the bird felt more responsive and easier to recover with. Collision handling was improved so bullets interacted with enemies more reliably. The health display was also made clearer so players could better track their remaining health during gameplay. These changes improved both functionality and overall playability.

To support gameplay problems with technical evidence, the Unity Test Runner, debug logs, and frame-by-frame analysis were used during development. These tools were used to verify C# logic, track collision behaviour, and assess control responsiveness.

The prototype also evolved from a basic *Flappy Bird* inspired game into a more complex action game. One of the most important changes made during evaluation was to the collision and damage system. Initially, every collision caused instant death, which made the heart

system ineffective. The design was revised so that obstacles caused immediate game over, while enemies only reduced player health. This produced a more balanced and rewarding gameplay experience.

5.2 Report on Final Testing

Final testing was used to evaluate whether the prototype met the intended gameplay and technical requirements. This included both playtesting and formal testing of the game logic.

5.2.1 Playtesting

Playtesting was used to assess gameplay feel, difficulty, clarity, and overall player experience. It helped identify whether the controls felt responsive, whether hazards were understandable, and whether the health and scoring systems contributed positively to the game. For playtesting results, please refer to the playtest document.

5.2.2 Unit and Integration Testing

In addition to playtesting, technical testing was used to verify that the core systems worked correctly. The Unity Test Framework was used to test key logic such as health reduction, score updates, difficulty scaling, player movement, and collision detection. Unit tests were used for isolated logic, while integration tests were used for behaviour that depended on Unity's physics system. Table V summarizes the main validation tests and the extent to which the prototype satisfied each requirement. Representative Unity Test Runner results and example test files are provided in Appendix A.

Table V Prototype Validation and Acceptance Testing

ID	Feature Tested	Testing Method	Acceptance Criteria	Results / Extent of Satisfaction
PB-07	Flap Mechanic	BirdFlap_ChangesVelocity (PlayMode)	Rigidbody2D must show positive vertical velocity change immediately after input.	Fully Satisfied: Vertical Y-position increased after gravity-balanced lift was applied.
PB-14	Health System	TakeDamage_DecrementsHealth (Unit Test)	The health variable must decrease by exactly the damage amount passed to the method.	Fully Satisfied: Initial health of 3 successfully reduced to 2 upon simulated enemy contact.
PB-18	Score Tracking	UpdateScore_IncrementsValue (Unit Test)	The internal score counter must increase by 1 and update the TextMeshPro UI string.	Fully Satisfied: _score field incremented correctly and UI was updated via Singleton instance.
PB-12	Collision Logic	PipeCollision_TriggersGameOver (Integration)	Collisions with "Obstacle" tagged objects	Fully Satisfied: Game state successfully

			must set Time.timeScale to 0.	transitioned to paused (0f) upon pipe impact.
PB-22	Difficulty Scaling	DifficultyScaling_Speed Check (Unit Test)	The speedPipe variable must increase by the designated multiplier (1.1x) correctly.	Partially Satisfied: Mathematical logic validated (0.65 to 0.715), though playtests suggest higher multipliers.
UI-01	Heart HUD	AddHeart_IncreasesCount (Unit Test)	Adding a heart must instantiate a new heart prefab as a child of the HeartManager.	Fully Satisfied: Transform child count increased to 1 after heart power-up simulation.

Because many variables were private, C# Reflection was used where necessary to inspect internal state changes during testing without altering the class structure. Unit tests were run in EditMode for pure logic, and integration tests were run in PlayMode for physics-based behaviour such as flapping and collisions. These tests helped confirm that the core mechanics worked as intended and reduced the risk of future changes breaking existing functionality.

Test Organization

The project used both EditMode and PlayMode tests, depending on whether the feature being tested relied on pure logic or Unity engine behaviour. Table VI summarizes how the tests were organized.

Table VI Test Types and Their Locations in Unity

Test Type	Tab	Why?
Unit Tests (e.g., ScoreTests, HeartTests)	EditMode	Used to test pure C# logic. These tests run quickly because they do not require the game engine to be running.
Integration Tests (e.g., MovementTests)	PlayMode	Used to test physics, coroutines, and other gameplay behaviour that depends on the game engine running.

Together, these tests provided both behavioural and functional evidence that the prototype met its main technical requirements, while the appendix contains supporting screenshots of representative test results.

5.3 Environmental, Societal, Safety, and Economic Considerations

This section considers broader factors beyond technical performance, including player safety, accessibility, cost, and environmental impact.

5.3.1 Risk and Safety Analysis

Developing interactive software involves identifying potential issues that could affect both system performance and player experience.

Design hazards:

- **Unfair obstacle generation:** Procedural generation could unintentionally create obstacle patterns that are too difficult or impossible to avoid, leading to player frustration. This risk was reduced by using randomized bounds that maintain a navigable gap.
- **Logic failure:** Reliance on real-time physics can lead to issues such as missed collisions or object overlap, including cases where the player or bullets pass through objects unexpectedly.

Safety and safeguards:

- **Photosensitivity:** The design avoids rapid flashing lights or abrupt visual transitions that could make gameplay uncomfortable for some players.
- **Data security:** The prototype runs locally on standard desktop hardware and does not collect sensitive personal information, so the risk of data breaches is minimal.

Warnings and training:

- **Player onboarding:** The entry screen provides a hovering idle state, giving players a moment to adjust to the game before starting.
- **Future improvement:** Later versions could include on-screen prompts or a short tutorial to help players learn flying and shooting controls more quickly.

5.3.2 Economic Considerations and Budgeting

The economic feasibility of this project is based on the use of accessible development tools and low-cost resources.

- **Budget planning:** The main resource required for the project was development time across five iterative sprints.
- **Cost analysis:** The project was completed with a \$0 direct budget by using the Unity Personal license and free assets.
 - All visual assets, sounds, and platforms used were free.
- **Resource sourcing:** Assets and development tools were obtained through free or free-to-use sources, allowing the project to remain within the original zero-cost budget.

5.3.3 Sustainability, Ethics, and Environmental Impact Analysis

The final design aligns with the project goals of accessibility, efficient resource use, and a low environmental footprint.

- **Ethical impact:** By prioritizing clear visual cues and simple controls, the game aims to remain accessible to players with different experience levels.
- **Life-cycle analysis (3 R's):**
 - **Reduce:** Using a lightweight 2D game reduces computational demands compared with more resource-intensive 3D systems.
 - **Reuse:** Development reused modular scripts and free assets where possible to reduce duplicated work.
 - **Recycle:** The Git-based repository preserves the codebase so it can be reused, revised, or extended in future projects.

- **Impact mitigation:** Designing for standard desktop hardware improves accessibility by avoiding the need for expensive or specialized devices.
- **Disposal:** As a digital product, the project does not create physical waste at the end of its use. Long-term storage or archiving can be handled through the project repository.

5.4 Limitations

Flap-and-Fight has several limitations. While the game demonstrates the core mechanics and overall design approach, some areas would need further testing, refinement, and expansion before the project could be considered a shippable product.

- **Limited playtesting scope:** Playtesting involved a limited number and variety of participants. As a result, there is only limited evidence regarding long-term engagement, accessibility for new players, and overall gameplay balance across different user groups.
- **Lack of onboarding guidance:** The prototype does not include a tutorial, start-screen instructions, or in-game prompts explaining actions such as flying and shooting. This may make the game harder for first-time players to learn and increase early confusion.
- **Spawning edge cases:** Although the enemy spawning system was refined multiple times, occasional overlap between static enemies and pipes was still observed. This suggests that the spawning logic works in most cases but is not yet fully robust under all gameplay conditions.
- **Desktop-only implementation:** The game was developed and tested only for desktop systems. While this meets the current project scope, the short-session style of gameplay would also suit mobile play, which was not explored in this iteration.
- **Limited gameplay variety:** The prototype currently includes only static and dynamic enemy types, with limited variation in hazards and interactions. Additional enemies, mechanics, or progression systems would be needed to provide greater gameplay depth.
- **Prototype-level polish:** Visual, particle, and audio effects are present, but they are not yet fully polished or cohesive. The project prioritized functional implementation over presentation, so the current prototype does not yet reflect the quality of a finished product.
- **Maintainability concerns:** While the system was organized into separate gameplay scripts, some code could still benefit from further cleanup, refactoring, and documentation to better support future maintenance and expansion.
- **Difficulty tuning limitations:** Infinite progression, randomized spawning, and physics-based interaction make precise control over pacing and difficulty more challenging. Although these systems improve replayability, they can also create inconsistent challenge levels between play sessions.
- **Time-related development constraints:** Development took place within a limited academic schedule and was affected by competing course demands. This reduced the time available for additional testing, balancing, polish, and feature refinement.

6 Team Work

This section outlines how the team collaborated throughout the project, including how work was divided, how decisions were made, and how progress was tracked across each sprint. The meeting summaries document the iterative development process by recording key discussions, completed work, and next tasks for each team member.

6.1 Meeting 1

Time: February 27, 2026, 1:30 pm to 2:30 pm

Present: Ruth, Shaylee

Agenda: Discuss game concept and determine next steps for the project.

Meeting minutes:

- Game concept discussed and agreed on.
- Core gameplay mechanics identified.
- Next step determined: set up development environment and create project backlog.

Team Member	Previous Task	Completion State	Next Task
Shaylee	N/A	N/A	Discuss and define initial game concept; define core gameplay mechanics
Ruth	N/A	N/A	Discuss and define initial game concept; define core gameplay mechanics

6.2 Meeting 2

Time: March 1, 2026, 11:30 am to 2:30 pm

Present: Ruth, Shaylee

Agenda: Setup development environment and organize project backlog.

Meeting minutes:

- The Unity project and GitHub repository were successfully created and configured.
- Cross-platform collaboration between Mac and Windows systems was verified.
- Git large file handling was researched and Git LFS was configured to support Unity asset management.
- The initial product backlog and user stories were finalized.
- A GitHub Kanban project board was created and backlog items were organized for tracking development tasks.
- Initial character and environment assets were researched to support future implementation.

Team Member	Previous Task	Completion State	Next Task
Shaylee	Discuss and define initial game concept; define core gameplay mechanics	100%	Create Unity project, configure GitHub repository, configure Mac/Windows compatibility, research Git LFS, create Kanban

			board, complete sprint documents
Ruth	Discuss and define initial game concept; define core gameplay mechanics	100%	Organize backlog items on Kanban board, research character assets, research environment assets

6.3 Meeting 3

Time: March 8, 2026, 1:00 pm to 2:00 pm

Present: Ruth, Shaylee

Agenda: Begin implementation of player movement and core environment systems.

Meeting minutes:

- Player flap movement was discussed and agreed on.
- Vertical movement and horizontal auto-movement were implemented.
- The flap/lift mechanic and supporting bird animation were added.
- The team agreed to continue refining movement feel in the next work session.

Team Member	Previous Task	Completion State	Next Task
Shaylee	Create Unity project, configure GitHub repository, configure Mac/Windows compatibility, research Git LFS, create Kanban board, complete sprint documents	100%	Complete Section 2 of the design report
Ruth	Organize backlog items on Kanban board, research character assets, research environment assets	100%	Implement vertical and horizontal movement; add flap/lift mechanic; add bird flap animation

6.4 Meeting 4

Time: March 13, 2026, 12:00 pm to 1:00 pm

Present: Ruth, Shaylee

Agenda: Implement gravity and review continuous movement behaviour.

Meeting minutes:

- Gravity was implemented for the player character.
- The play scene was set to move horizontally on a continuous basis.
- The team reviewed movement behaviour and agreed to move next to obstacle spawning and collision

Team Member	Previous Task	Completion State	Next Task
Shaylee	Complete Section 2 of the design report	100%	Complete section 3 of the design report; update sprint 2 document
Ruth	Implement vertical and horizontal movement; add flap/lift mechanic; add bird flap animation	100%	Implement gravity system; implement continuous horizontal scene movement; prepare obstacle spawning system

6.5 Meeting 5

Time: March 20, 2026, 2:00 pm to 3:00 pm

Present: Ruth, Shaylee

Agenda: Implement obstacle generation and collision behaviour for the first playable prototype.

Meeting minutes:

- Obstacles were made to appear infinitely at randomized positions.
- Collision with pipes was implemented as an immediate-death condition.
- The team confirmed that a basic playable prototype had been achieved.
- It was noted that obstacle spacing and placement would still need refinement in a later sprint.

Team Member	Previous Task	Completion State	Next Task
Shaylee	Complete section 3 of the design report; update sprint 2 document	100%	Complete sprint 2 document; update teamwork
Ruth	Implement gravity system; implement continuous horizontal scene movement; prepare obstacle spawning system	100%	Complete sprint 2 document; refine concepts for three formal alternative design solutions

6.6 Meeting 6

Time: March 26, 2026, 7:00 pm to 8:00 pm

Present: Ruth, Shaylee

Agenda: Refine obstacle behaviour and begin enemy and attack system design.

Meeting minutes:

- Obstacles were updated to generate randomly with more consistent spacing.
- Obstacle movement toward the player was implemented.
- Stationary enemy generation was started.
- The team discussed the basic attack mechanic and decided to use bullets.

Team Member	Previous Task	Completion State	Next Task
Shaylee	Complete sprint 2 document; update teamwork	100%	Refine obstacle spawning rules; begin score system planning; implement stationary enemy generation and enemy interaction logic
Ruth	Complete sprint 2 document; refine concepts for three formal alternative design solutions	100%	Complete section 4 of the design report; update sprint document; update teamwork

6.7 Meeting 7

Time: March 27, 2026, 2:00 pm to 5:00 pm

Present: Ruth, Shaylee

Agenda: Implement combat interaction and scoring, and review incomplete sprint items.

Meeting minutes:

- The basic attack mechanic was implemented using bullets shot from the player.
- Stationary enemies were set to die when hit by the player attack.
- A score system was implemented for passing between pipes.
- The team discussed incomplete items that would return to the backlog.
- Score display placement near the top of the screen beside the best score was also discussed.

Team Member	Previous Task	Completion State	Next Task
Shaylee	Refine obstacle spawning rules; begin score system planning; implement stationary enemy generation and enemy interaction logic	100%	Implement enemy damage behaviour; test enemy attack interaction; complete sprint 3 document; update teamwork
Ruth	Complete section 4 of the design report; update sprint document; update teamwork	100%	Implement bullet attack mechanic; implement enemy hit behaviour for stationary enemies; implement pipe-passing score system; review incomplete items for backlog transfer

6.8 Meeting 8

Time: April 3, 2026, 2:00 pm to 3:00 pm

Present: Ruth, Shaylee

Agenda: Implement user interface features, health display, difficulty scaling, and power-ups.

Meeting minutes:

- A game over screen was implemented using a free online sprite resource.
- A heart-based health display was added so the player can track remaining lives.
- Difficulty scaling was implemented through changes to pipe and enemy generation and movement speed.
- Two power-ups, heart and shield, were implemented.
- An entry screen with the player hovering was also added.
- The glide feature was not completed and was moved to the next sprint.

Team Member	Previous Task	Completion State	Next Task
Shaylee	Implement enemy damage behaviour; test enemy attack interaction; complete sprint 3 document; update teamwork	100%	Implement entry screen; update teamwork, complete sprint 4 document
Ruth	Implement bullet attack mechanic; implement enemy hit behaviour for stationary enemies; implement pipe-passing score system; review incomplete items for backlog transfer	100%	Implement game over screen; implement health display HUD; implement difficulty scaling; implement heart and shield power-ups; complete sprint 4 document

6.9 Meeting 9

Time: April 9, 2026, 7:00 pm to 8:30 pm

Present: Ruth, Shaylee

Agenda: Add final enhancing features and review remaining limitations.

Meeting minutes:

- The team reviewed the newly added combat feedback and enemy behaviour in the integrated prototype.
- The hold-to-glide feature was discussed and remained difficult to balance with the existing gravity system.
- Final next steps were identified as integrated testing, glide review, and final documentation updates.

Team Member	Previous Task	Completion State	Next Task
Shaylee	Implement entry screen; update teamwork, complete sprint 4 document	100%	Implement hit feedback effects; implement music and sound effects; implement aggressive dive enemy behaviour; test moving enemy interaction; update teamwork
Ruth	Implement game over screen; implement health display HUD; implement difficulty scaling; implement heart and shield power-ups; complete sprint 4 document	100%	Add prefab to the shield power-up; test final integrated features; complete sprint 5 document; complete section 5 of the design report

Overall, the team worked together effectively and maintained good communication throughout development. Since both team members have collaborated on many projects, a strong level of trust had already been established. This made it easier to divide tasks, adapt when needed, and communicate openly about outstanding work. Both team members stayed committed to completing the project, which helped keep the workflow flexible, dependable, and efficient.

Scores	Team member 1	Team member 2
Name	Shaylee	Ruth
Student ID	T00551934	T00696672
Total mark	100 %	100%
Notes and Comments		

7 Project Management

Project management was used to coordinate gameplay implementation, testing, documentation, and team responsibilities. The schedule reflects the progression from initial setup and planning to prototyping, refinement, validation, and final integration.

Table VII summarizes the major project milestones used to track progress across the five sprints.

Table VII Project Milestone Schedule

Milestone	Target date	Description
Project foundation complete	March 1, 2026	Unity project, GitHub repository, Git LFS, backlog, and Kanban board established
First playable prototype complete	March 20, 2026	Core movement, gravity, obstacle spawning, and pipe collision implemented
Combat systems complete	March 27, 2026	Basic attack, stationary enemies, and score system implemented
UI and gameplay flow complete	April 3, 2026	Entry screen, game over screen, health HUD, power-ups, and difficulty scaling implemented
Final prototype complete	April 10, 2026	Hit feedback, aggressive enemy, final testing, and final report integration completed

Table VIII summarizes the primary responsibility distribution for major tasks and milestones. Since this was a small two-person project, some responsibilities were shared, while others were led by one team member and supported by the other.

Table VIII Responsibility Assignment Matrix

Task / Milestone	Shaylee	Ruth
Game concept and core mechanics	Shared	Shared
Unity, GitHub, Git LFS, and Kanban setup	Lead	Support
Asset research	Support	Lead
Core movement, gravity, and obstacle prototype	Support	Lead
Obstacle refinement and enemy interaction logic	Lead	Support
Basic attack, enemy hit behaviour, and score system	Support	Lead

Entry screen and gameplay presentation features	Lead	Support
HUD, game over screen, difficulty scaling, and power-ups	Support	Lead
Hit feedback, sound integration, and moving enemy testing	Lead	Support
Sprint documentation	Shared	Shared
Design report writing and editing	Shared	Shared
Final integrated testing and review	Support	Lead

Figure 5 shows the overall project schedule across the five sprints, including major tasks, milestones, and overlapping implementation and documentation work. The chart reflects the agile nature of the project, since gameplay systems were developed in stages and then refined through later testing and integration.

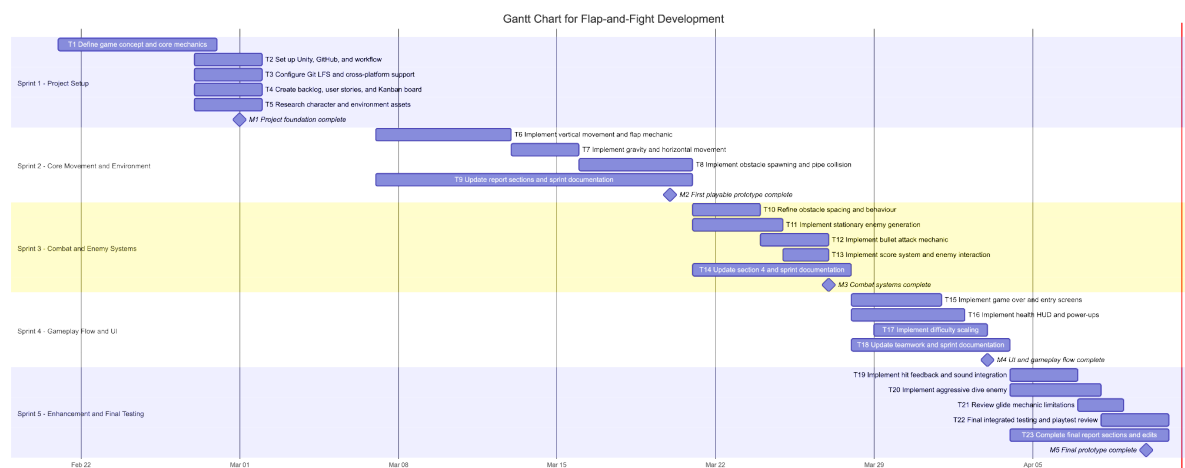


Figure 7. Gantt chart for *Flap-and-Fight* development.

The Gantt chart shows that implementation, testing, and reporting often occurred in parallel rather than as separate phases. Early work focused on project setup and planning, followed by core movement and environment systems, then combat and enemy systems, then UI and gameplay flow features, and finally enhancement, testing, and final report integration.

Slack time was limited for most implementation-heavy tasks, particularly during Sprints 2 through 5, where gameplay features were developed in small increments and depended on earlier systems. Small scheduling buffers existed mainly in documentation and reporting tasks, which could overlap with implementation work. As a result, core gameplay systems such as movement, obstacle spawning, combat, UI integration, and final testing operated with low slack, while sprint documentation and report writing provided some flexibility in the schedule.

Project management was effective overall because the work was divided into manageable sprint goals and supported by a product backlog, sprint planning, and regular progress

tracking. Early setup of the Unity project, GitHub repository, and Kanban board created a solid foundation for the rest of development.

One area that could be improved in future projects is task estimation and breakdown, especially for features involving physics tuning, balancing, and polish. Some features were simpler to implement, while others required more iteration than expected and were carried into later sprints. Future work would benefit from smaller task definitions, more time reserved for playtesting and refinement, and clearer tracking of a few later-stage implementation items. Even so, the project schedule remained workable, and the team was able to complete the main prototype goals within the available timeframe.

8 Conclusions

The development of *Flap-and-Fight* resulted in a functional side-scrolling action game prototype that combines movement, obstacles, combat, scoring, health, and visual feedback into a single playable system. Over five sprints, the project progressed from initial planning and environment setup to implementation, testing, refinement, and final integration.

The final physics-based solution was appropriate for the project because it provided the responsiveness and gameplay feel needed for a fast-paced game. Core systems such as flap movement, gravity, obstacle spawning, collision handling, enemy interaction, scoring, HUD elements, and game state management were implemented successfully. Together, these features addressed the main design problem by producing a playable game with intuitive controls, recognizable hazards, and a clear gameplay loop.

The project was also feasible within the course scope. Unity provided a practical platform for building and testing the prototype, while GitHub and sprint-based planning supported collaboration and version control throughout development. The project remained within a zero-cost software budget by using Unity Personal and free assets, making the prototype realistic to complete within the available time and resources.

From a stakeholder perspective, the final design addressed many of the identified needs. Players were given simple controls, visible health and score feedback, and a more balanced experience through the separation of obstacle death and enemy damage. Developer and maintainer needs were also considered through the use of modular scripts, version control, and automated testing of several core systems. At the same time, the project still has limitations, particularly in playtesting scope, tutorial instructions, gameplay variety, and final polish.

One of the main lessons from the project was that many design decisions could not be finalized through planning alone. Features such as movement feel, difficulty balance, collision behaviour, and enemy interaction had to be adjusted through repeated testing. Thus, building and refining the game in iterative stages made it easier to identify issues early and improve the prototype over time.

Overall, *Flap-and-Fight* met the main goals of the project and produced a solid prototype within the available timeframe. While further balancing, playtesting, and polish would still be needed before release, the project produced a workable foundation for future development.

9 Future Works

Future development should focus on expanding gameplay variety and improving overall polish. The following areas were identified as priorities for future work:

Glide Mechanic

- PB-09 implementation: Finalize the hold-to-glide feature to give players more precise control when moving through narrow obstacle gaps.
- Accessibility benefit: This could reduce the need for constant tapping and make the controls easier for some players to use.

Improve Dynamic Enemy AI

- Tracking behaviour: In addition to the aggressive dive behaviour, a chase enemy could be added to follow the player's vertical position more consistently.
- Gameplay impact: This would encourage more active combat instead of relying mostly on avoidance.

Environment Variation

- Environment changes: Introduce different settings, such as a forest, cave, or sky city, that change as the score increases.
- Gameplay variety: Each environment could add different conditions, such as altered gravity or wind resistance, to create more variation between runs.

Expand Weapon Power-Ups

- Weapon variety: Expand the projectile system to include different weapon types, such as a spread shot for multiple enemies or a laser for clearing a vertical path.
- Player choice: Different weapons could support different playstyles depending on the hazards on screen.

Visual and Audio Polish

- Graphics: Replace placeholder free assets with more polished sprites that follow a consistent art style.
- Audio: Add music that changes with the pace or difficulty of the game to improve feedback and atmosphere.

10 References

[1] C. Fabricatore, "Gameplay and Game Mechanics Design: A Key to Quality in Videogames." Available:

https://www.researchgate.net/publication/236168267_Gameplay_and_game_mechanics_design_a_key_to_quality_in_videogames

[2] S. Wright et al., "Exploring player experiences with moral decision making mechanics in video games," Computers in Human Behavior, 2024. Available:

<https://www.sciencedirect.com/science/article/pii/S0747563224002929>

Appendix A. Representative Test Results

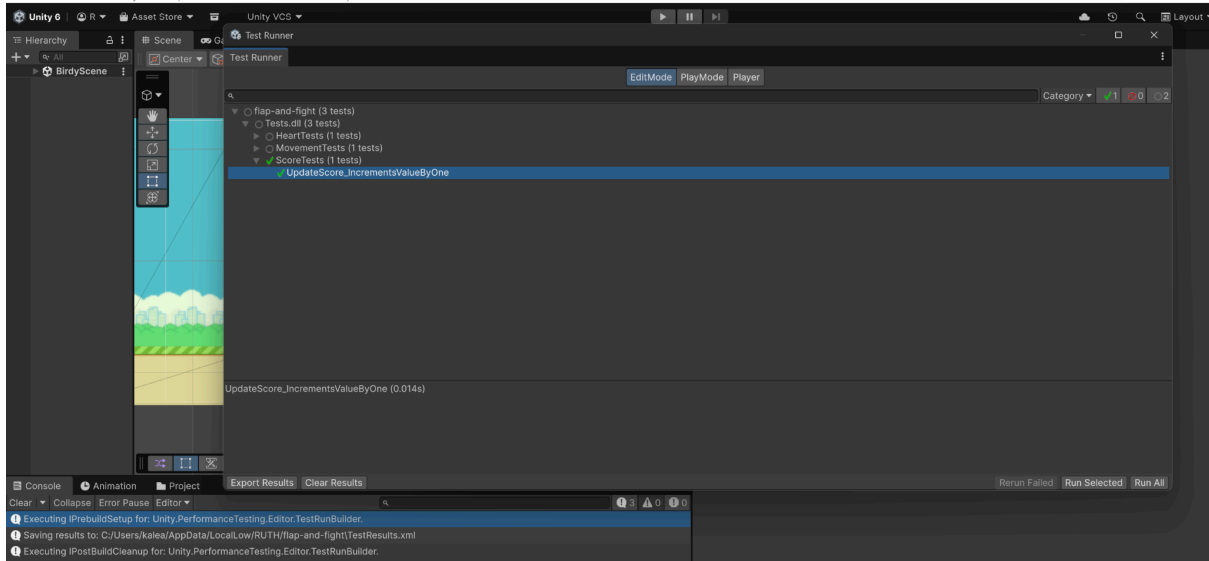


Figure A1. Unity Test Runner results for *ScoreTests.cs* unit test.

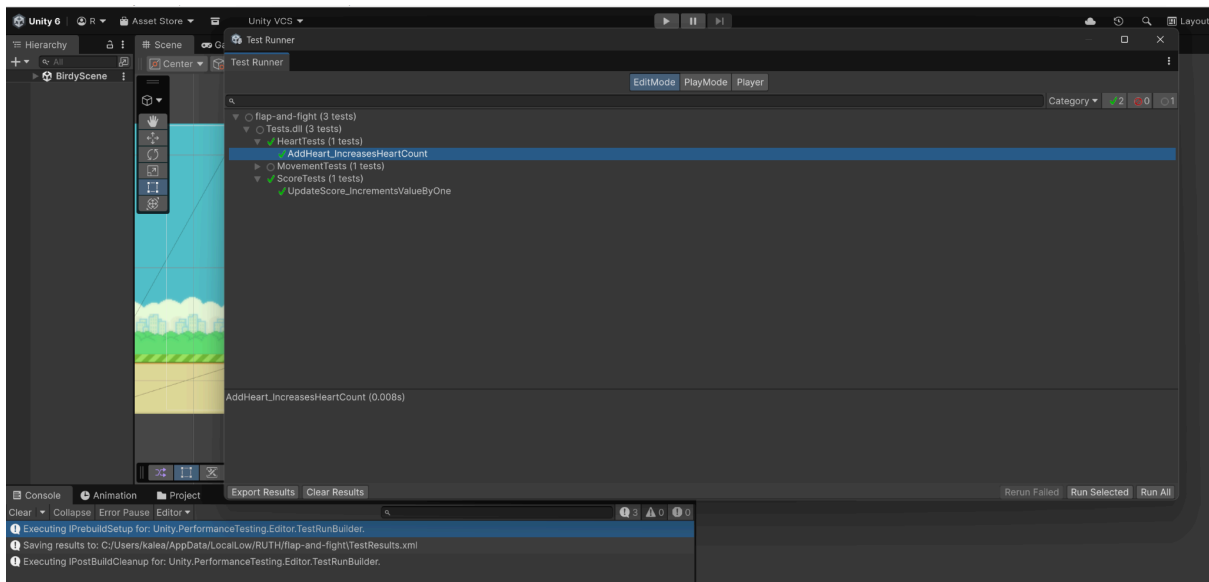
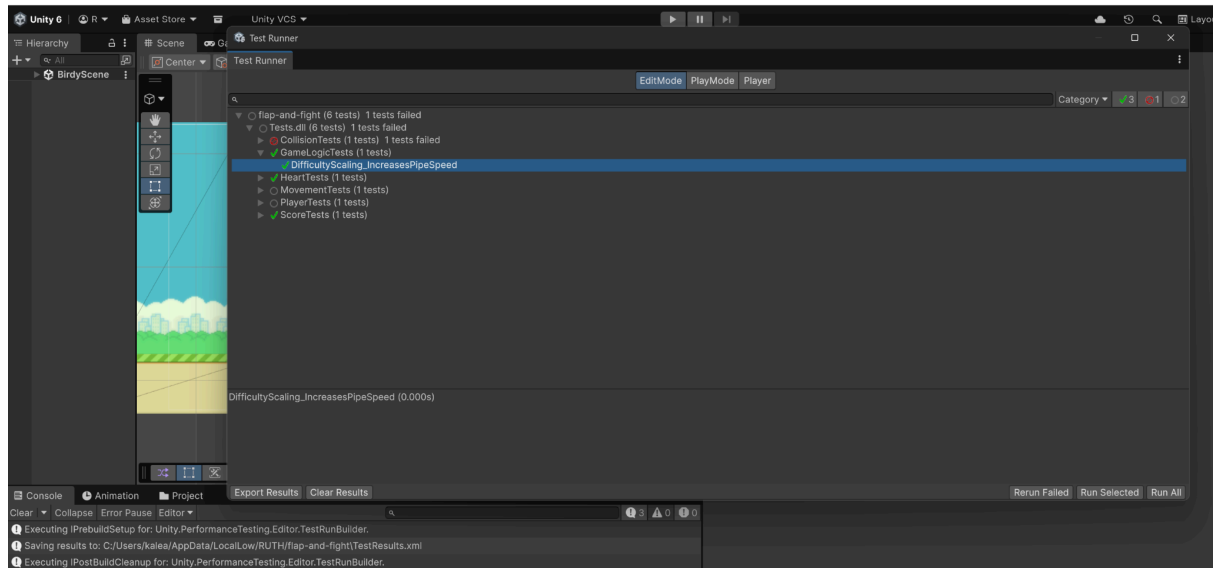
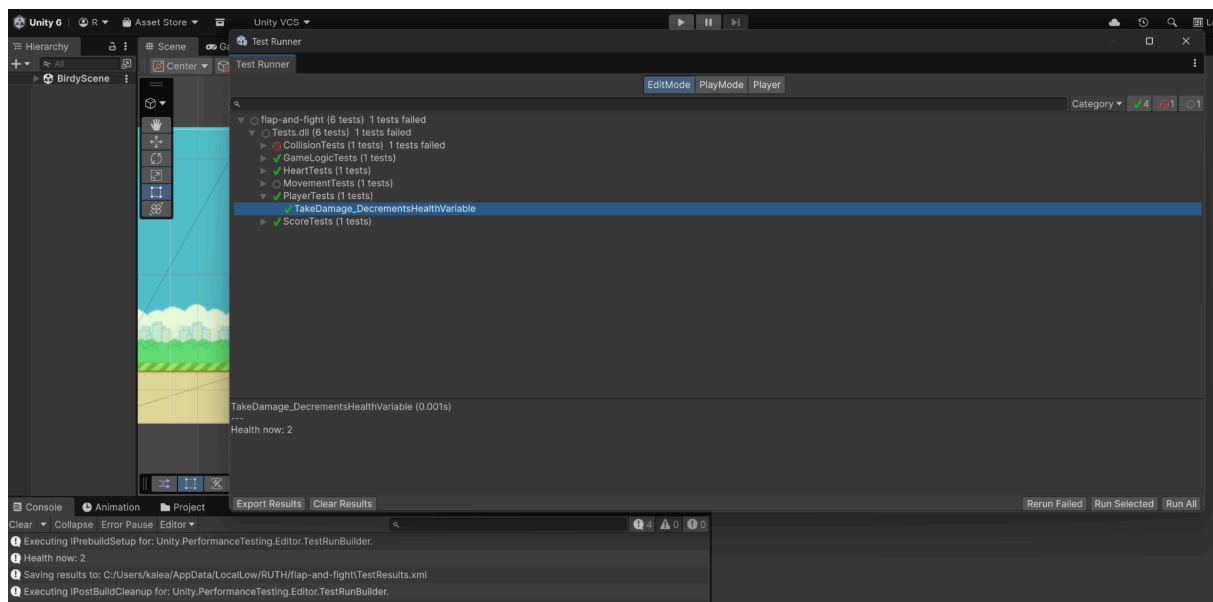


Figure A2. Unity Test Runner results for *HeartTests.cs* unit test.

Figure A3. Unity Test Runner results for *GameLogicTests.cs* unit test.Figure A4. Unity Test Runner results for *PlayerTests.cs* unit test.

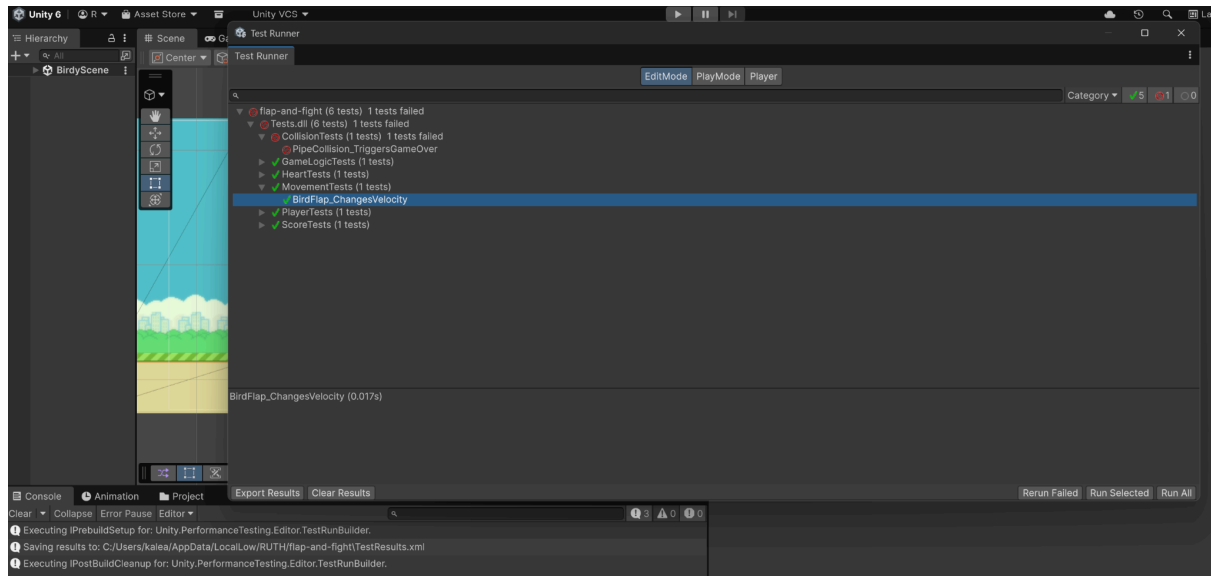


Figure A5. Unity Test Runner results for *MovementTests.cs* integration test.

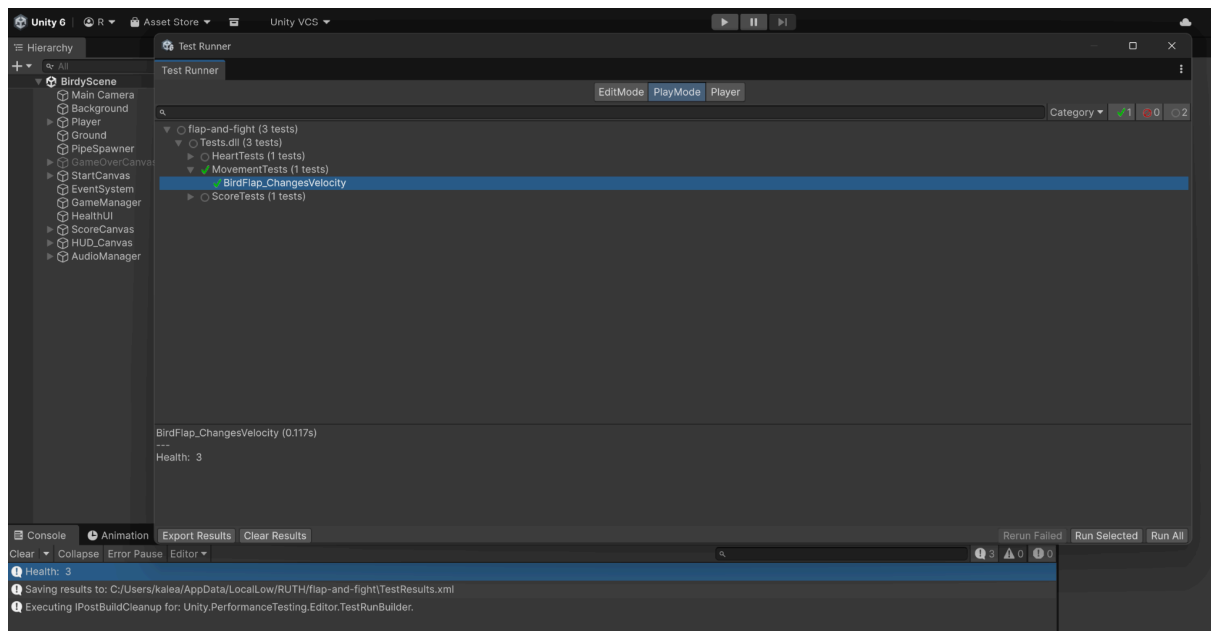


Figure A6. Unity Test Runner results for *MovementTests.cs* integration test.

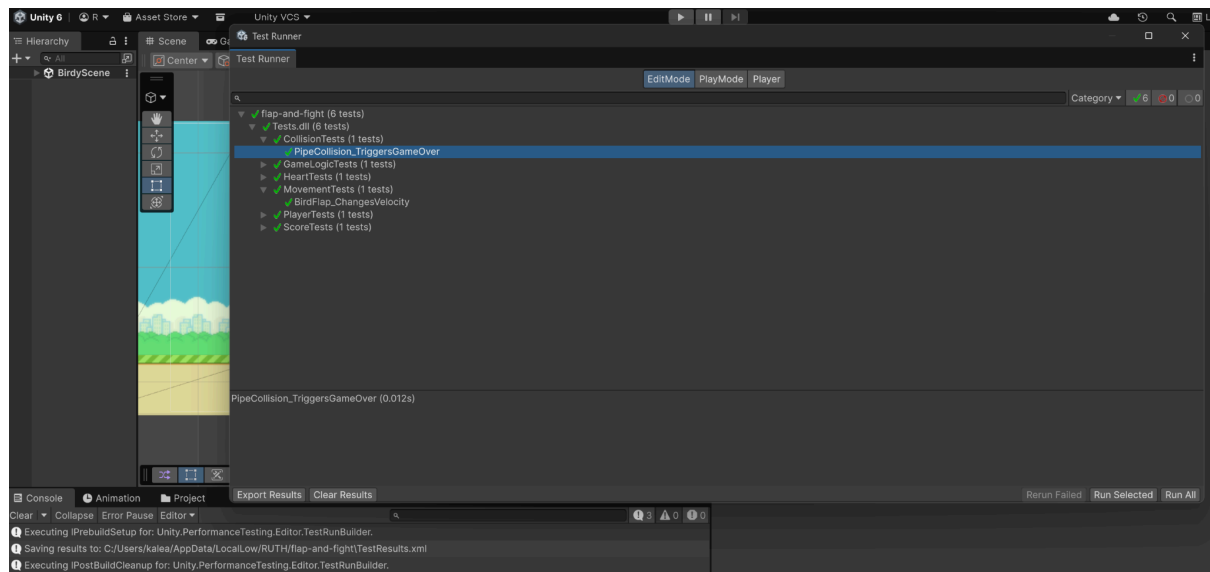


Figure A7. Unity Test Runner results for *CollisionTests.cs* integration test.